

Github: <https://github.com/movchun567/apz-kravchuk.git> , branch micro_basics

logging-service реалізовано як простий Flask вебзастосунок, який приймає транзакції через HTTP POST і зберігає їх лише в оперативній пам'яті. У logging-service всі операції запису в словники (збереження транзакції та оновлення списку транзакцій користувача) виконуються всередині with lock:. Це означає, що в один момент часу лише один потік може змінювати спільні структури даних.

counter-service також реалізовано на Flask і зберігає баланси користувачів в оперативній пам'яті як logging-service. У counter-service блокування використовується під час оновлення балансу. Операція читання поточного балансу, додавання суми та запис нового значення виконується як одна атомарна критична секція під Lock.

У facade-service використано ThreadPoolExecutor для паралельного виконання HTTP-викликів до logging-service та counter-service. Під час обробки кожного POST-запиту facade має відправити транзакцію одночасно в два сервіси: один зберігає її в пам'яті, інший оновлює баланс користувача. Якщо виконувати ці виклики послідовно, то загальний час обробки запиту дорівнював би сумі часу звернення до logging-service і counter-service.

Щоб зменшити затримку, було використано ThreadPoolExecutor. У межах одного запиту facade створює дві задачі в пулі потоків: одну для виклику logging-service, іншу для виклику counter-service. Ці задачі виконуються паралельно, після чого facade очікує їх завершення та формує відповідь клієнту. У результаті сумарна латентність одного запиту приблизно дорівнює максимальному часу одного з викликів, а не їх сумі.

Щоб запускати всю систему одразу, будемо її в docker контейнері, де окремо ініціалізуємо кожен мікросервіс:

```
Attaching to counter-service-1, facade-service-1, logging-service-1
logging-service-1 | [2026-02-22 20:05:45 +0000] [1] [INFO] Starting gunicorn 22.0.0
logging-service-1 | [2026-02-22 20:05:45 +0000] [1] [INFO] Listening at: http://0.0.0.0:8001 (1)
logging-service-1 | [2026-02-22 20:05:45 +0000] [1] [INFO] Using worker: sync
logging-service-1 | [2026-02-22 20:05:45 +0000] [7] [INFO] Booting worker with pid: 7
counter-service-1 | [2026-02-22 20:05:45 +0000] [1] [INFO] Starting gunicorn 22.0.0
counter-service-1 | [2026-02-22 20:05:45 +0000] [1] [INFO] Listening at: http://0.0.0.0:8002 (1)
counter-service-1 | [2026-02-22 20:05:45 +0000] [1] [INFO] Using worker: sync
counter-service-1 | [2026-02-22 20:05:45 +0000] [7] [INFO] Booting worker with pid: 7
facade-service-1 | [2026-02-22 20:05:45 +0000] [1] [INFO] Starting gunicorn 22.0.0
facade-service-1 | [2026-02-22 20:05:45 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
facade-service-1 | [2026-02-22 20:05:45 +0000] [1] [INFO] Using worker: sync
facade-service-1 | [2026-02-22 20:05:45 +0000] [7] [INFO] Booting worker with pid: 7
```

Потім тестуємо чи працюють різні запити:

Транзакції:

POST

▼

http://localhost:8000/transaction

Docs

Params

Authorization

Headers (9)

Body ●

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

```
1 {
2   "user_id": "user1",
3   "amount": 10
4 }
```

Body

Cookies (1)

Headers (5)

Test Results

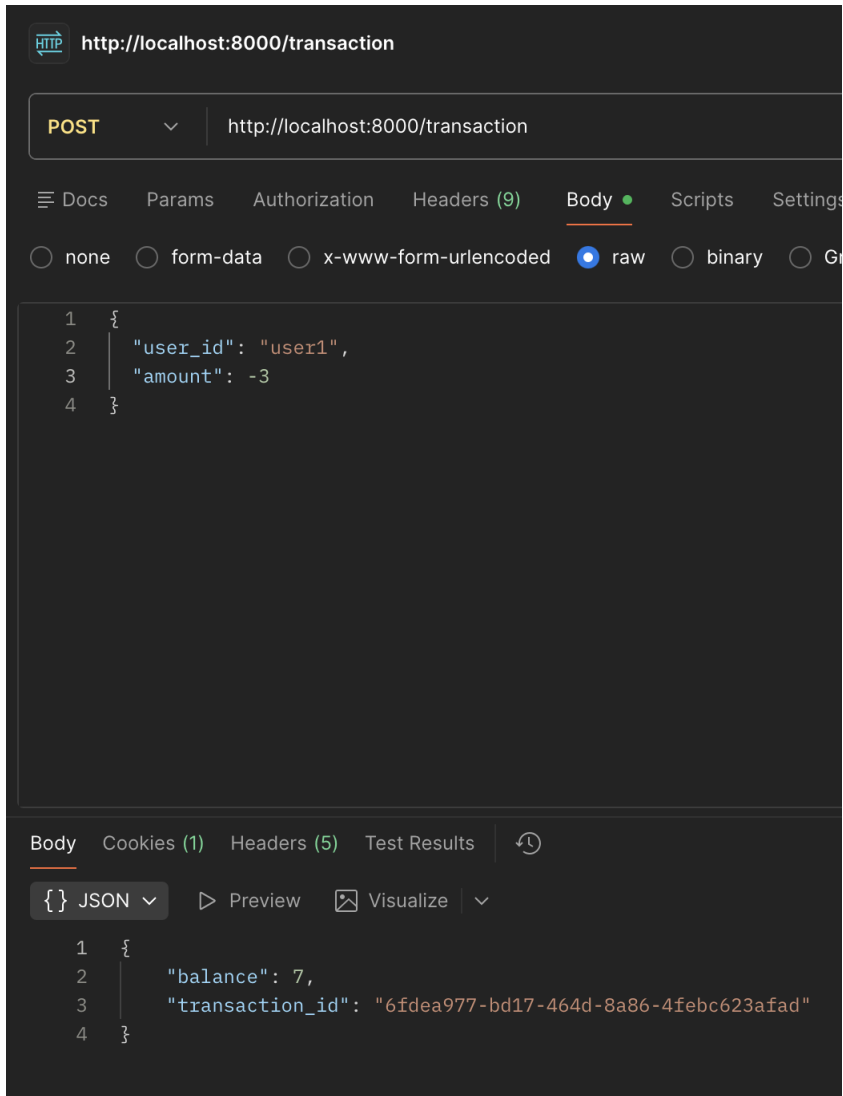
↺

{ } JSON ▼

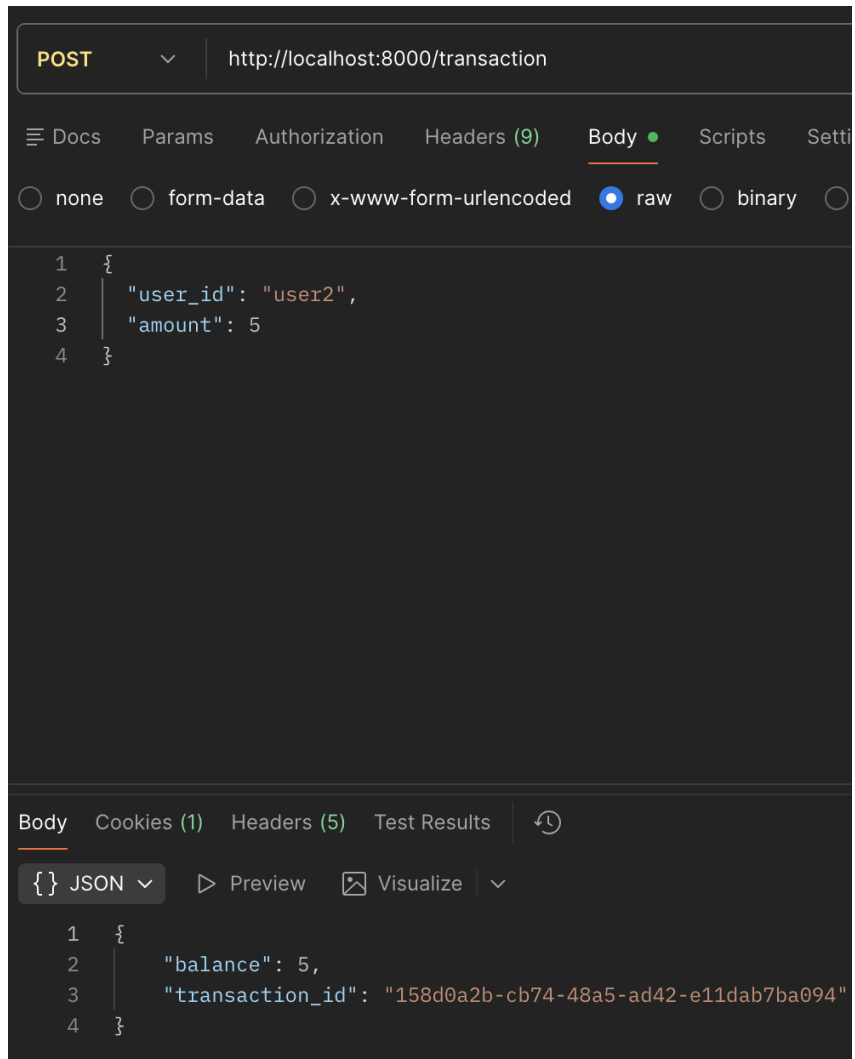
▶ Preview

🖼 Visualize ▼

```
1 {
2   "balance": 10,
3   "transaction_id": "d3352292-6804-48a3-b6b0-c798f34fab72"
4 }
```



Інший користувач:



Перевірити users:

GET

▼

http://localhost:8000/user/user1

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

☐ JSON

1

^P to Ask AI

Body

Cookies (1)

Headers (5)

Test Results

↺

{ } JSON ▼

▶ Preview

🖼 Visualize

▼

1

{

2

"balance": 7,

3

"transactions": [

4

{

5

"amount": 10,

6

"timestamp": 1771790897.5117223,

7

"transaction_id": "d3352292-6804-48a3-b6b0-c798f34fab72",

8

"user_id": "user1"

9

},

10

{

11

"amount": -3,

12

"timestamp": 1771791046.229103,

13

"transaction_id": "6fdea977-bd17-464d-8a86-4febc623afad",

14

"user_id": "user1"

15

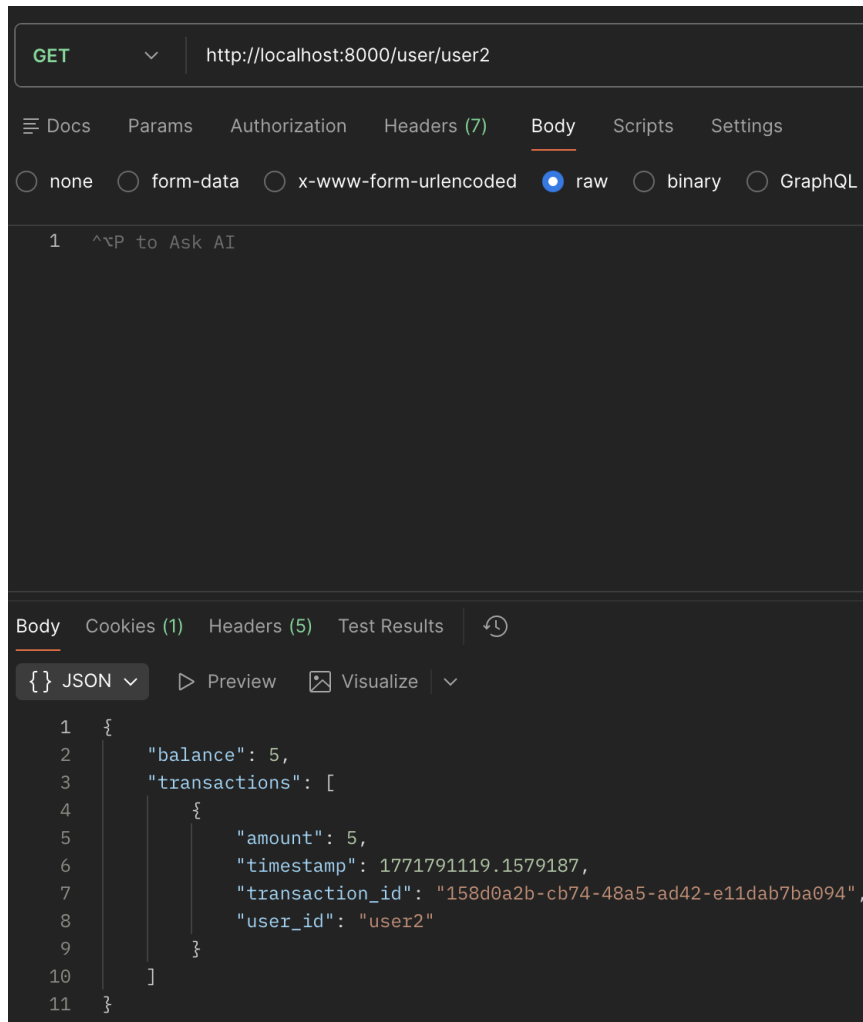
}

16

]

17

}



Баланси всіх акаунтів:

GET

▼

http://localhost:8000/accounts

☰ Docs

Params

Authorization

Headers (7)

Body

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒

1

^cP to Ask AI

Body

Cookies (1)

Headers (5)

Test Results

↺

{ } JSON ▼

▶ Preview

🖼 Visualize ▼

1

{

2

"balances": {

3

"user1": 7,

4

"user2": 5

5

}

6

}

Метрики:

GET ⌵ | http://localhost:8000/metrics

☰ Docs Params Authorization Headers (7) Body

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw

1 ^P to Ask AI

Body Cookies (1) Headers (5) Test Results ↺

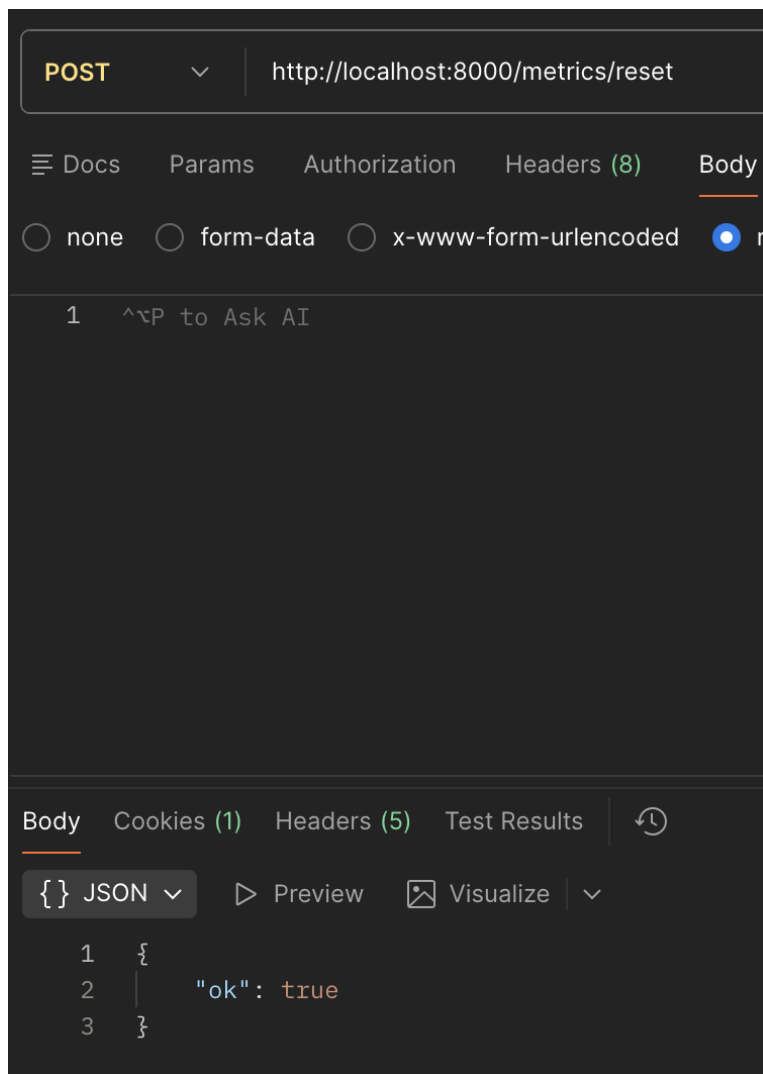
{} JSON ⌵ ▶ Preview 🖼️ Visualize ⌵

```
1 {
2   "counter_avg_ms": 3.311420833445785,
3   "counter_calls": 6,
4   "counter_time_sec": 0.01986852500067471,
5   "logging_avg_ms": 3.675618399756786,
6   "logging_calls": 5,
7   "logging_time_sec": 0.01837809199878393
8 }
```

- Logging_calls: Кількість викликів до logging-service.
- Counter_calls: Кількість викликів до counter-service.
- Logging_time_sec: Сумарний час (у секундах), витрачений на виклики logging-service.
- Counter_time_sec: Сумарний час (у секундах), витрачений на виклики counter-service.
- Logging_avg_ms: Середній час одного виклику до logging-service в мілісекундах.

- Counter_avg_ms: Середній час одного виклику до counter-service в мілісекундах.

Ресетимо метрики перед тестами:



Тести:

```

logging-service-1 | [logging-service] stored tx=a9d7c7d7-977f-41c9-9107-6f897af24840 user=shared amount=1
logging-service-1 | [logging-service] stored tx=00e1f06b-5172-4050-81ed-92f2f23b82c1 user=shared amount=1
logging-service-1 | [logging-service] stored tx=98c84b8f-6eec-4847-9485-46a947eb7f3a user=shared amount=1
logging-service-1 | [logging-service] stored tx=9602cfea-331c-494c-b87d-8a85c61d8858 user=shared amount=1
logging-service-1 | [logging-service] stored tx=1c54d318-41e2-4204-abd8-c5db1913083a user=shared amount=1
logging-service-1 | [logging-service] stored tx=6c5603bc-1cc1-4851-a306-51ccc1411e93 user=shared amount=1
logging-service-1 | [logging-service] stored tx=6134f307-2b99-463c-b9f7-e07236a13995 user=shared amount=1
logging-service-1 | [logging-service] stored tx=6ccbae8d-53cb-41a8-89e0-7ea290feebb9 user=shared amount=1
logging-service-1 | [logging-service] stored tx=ffe3ebc1-4afc-4275-a9b7-adb186c29651 user=shared amount=1
counter-service-1 | [counter-service] user=shared 99808 -> 99809 (amount=1)
counter-service-1 | [counter-service] user=shared 99809 -> 99810 (amount=1)
counter-service-1 | [counter-service] user=shared 99810 -> 99811 (amount=1)
counter-service-1 | [counter-service] user=shared 99811 -> 99812 (amount=1)
counter-service-1 | [counter-service] user=shared 99812 -> 99813 (amount=1)
counter-service-1 | [counter-service] user=shared 99813 -> 99814 (amount=1)
counter-service-1 | [counter-service] user=shared 99814 -> 99815 (amount=1)
counter-service-1 | [counter-service] user=shared 99815 -> 99816 (amount=1)
counter-service-1 | [counter-service] user=shared 99816 -> 99817 (amount=1)
counter-service-1 | [counter-service] user=shared 99817 -> 99818 (amount=1)
counter-service-1 | [counter-service] user=shared 99818 -> 99819 (amount=1)
counter-service-1 | [counter-service] user=shared 99819 -> 99820 (amount=1)
counter-service-1 | [counter-service] user=shared 99820 -> 99821 (amount=1)

```

```

akravchuk@macA apz-kravchuk % python3 tests.py

=== Scenario 1 (10 users, each own account) ===
Total requests: 100000
Total time (sec): 336.276
Requests/sec: 297.4
Facade metrics: {'counter_avg_ms': 1.9534651647798094, 'counter_calls': 100000, 'counter_time_sec': 195.3465164779809, 'logging_avg_ms': 2.229788054494911, 'logging_calls': 100000, 'logging_time_sec': 222.97880544949112}

=== Scenario 2 (all to same account) ===
Total requests: 100000
Total time (sec): 342.584
Requests/sec: 291.9
Facade metrics: {'counter_avg_ms': 1.996572712178295, 'counter_calls': 100000, 'counter_time_sec': 199.65727121782948, 'logging_avg_ms': 2.29258331849951, 'logging_calls': 100000, 'logging_time_sec': 229.258331849951}

```

У першому сценарії, де 10 користувачів виконували по 10 000 транзакцій кожен на власний рахунок, система обробила 100 000 запитів за 336 секунд, що становить приблизно 297 запитів на секунду. Середній час виклику counter-service склав близько 1.95 мс, а logging-service — близько 2.23 мс. Це свідчить про стабільну роботу системи при паралельному навантаженні без конфліктів доступу до даних.

У другому сценарії всі 100 000 транзакцій виконувалися на один і той самий рахунок. Загальний час виконання становив 342 секунди, а продуктивність знизилася до приблизно 292 запитів на секунду. Невелике зниження швидкості пояснюється конкуренцією потоків за блокування (Lock) у counter-service, оскільки всі операції змінювали один баланс. При цьому середній час викликів залишився майже таким самим, що підтверджує відсутність race condition і коректність реалізації синхронізації.

Загалом система демонструє стабільну продуктивність, а основні витрати часу пов'язані з HTTP-взаємодією між мікросервісами.